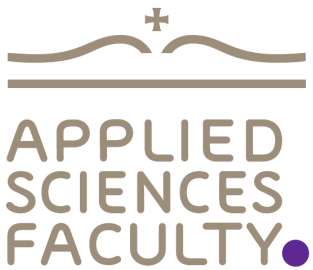
C++ — лекція 25. Лямбда-функції, constexpr та <functional>





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

constexpr — узагальнені константні вирази

- Для змінних constexpr — "підсилена" форма const. Вона вимагає, щоб значення було обчислено під час компіляції.
- Звичайні const-змінні, навіть якщо технічно їх можна знайти під час компіляції, на розсуд компілятора, можуть ініціалізуватися вже під час виконання[1].
- Крім того, константи можна ініціалізувати й змінними – тоді вони гарантовано не будуть відомими на момент компіляції, хоча їх і не можна буде змінювати в програмі:

```
int x;
cin >> x;
const int y = x;
//    constexpr int z = x;    //! Error
//    constexpr int z = y;    //! Error
```

[1] Особливо гостро це відчувається на мікроконтролерах — див. http://indrekis2.blogspot.com/2015/11/c-arm-gcc-stm32-coide.html, розділ "Ініціалізація глобальних змінних". Його якраз присвячено викликанню коду ініціалізації, зокрема і констант.

- Ініціалізатором для constexpr змінних можуть служити літерали, інші constexpr та результати constexpr-функцій.
- Їх тип має задовольняти вимозі (концепції) LiteralType – див. далі.
- Аргументами шаблонів – не типами, звичайно, мають бути константи компіляції. Це ж стосується розмірів вбудованих масивів[2].

```
const int y = x;

// array<int, y> ar1;      //! Error!
constexpr auto N = 5;
array<int, N> ar2;         // OK
```

[2] C99 дозволило вбудовані масиви змінного розміру. Однак, через опір розробників компіляторів, в C11 цю можливість оголосили опціональною. C++ її ніколи не включала — в жодному із стандартів. Однак, GCC, включає її за замовчуванням, навіть в педантичному режимі.

LiteralType

- LiteralType[3] — це:
  - скалярний тип,
  - або посилання на нього,
  - або агрегат LiteralType типу/типів,
  - або клас із тривіальним деструктором, який є або агрегатом або має хоча б один constexpr-конструктор, який не є конструктором копіювання або переміщення, а всі його не-статичні члени є LiteralType.
- Всі вбудовані типи, крім void — LiteralType.
  - C++14 додав і підтримку void.

[3] Детальніше див. http://en.cppreference.com/w/cpp/concept/LiteralType.

constexpr-функції

- Функції, помічені як constexpr, складніші та цікавіші.
- Може видатися дещо дивним, але результат таких функцій — не обов'язково константа. Вимагається, щоб існував **хоча б один** набір аргументів, для якого її значення можна розрахувати під час компіляції[4].
- Результати таких функцій можна використовувати в ролі констант часу компіляції — наприклад, для розміру масивів:

```
constexpr int doublificy (int x){
    return 2*x;
}

array<int, doublificy (N)> ar3;
// але
cout << doublificy(x) << endl;
```

[4] В C++23 дуже пом'якшили умови контролю компілятора за цим – для forward compatibility.

- В C++11 на тіло такої constexpr функції накладаються доволі жорсткі обмеження — в основному, вона має складатися з одного return. Хоча, завдяки тернарному оператору вибору, це не дуже суворе обмеження:

```
constexpr unsigned long cfactorial (unsigned long n){
    return n <= 1 ? 1 : (n * cfactorial (n - 1));
}
```

- Вимоги до тіла constexpr функції в C++11: **може містити** static_assert, typedef/using-синоніми, які не оголошують класів чи перерахувань, using-оголошення та директиви і рівно один return.
- C++14 різко пом'якшив ці вимоги: **заборонено** лише: асемблерні вставки, goto та мітки крім тих, що в switch, try-блоки, оголошення не LiteralType змінних, оголошення thread-local змінних та оголошення змінних без ініціалізації.
- **В будь-якому випадку**, вона не може бути віртуальною і повинна приймати та повертати лише LiteralType-типи.
- Constexpr функції є неявно inline, тому їх тіло повинно бути у всіх одиницях трансляції, де вони використовуються.
- Детальніше див. `http://en.cppreference.com/w/cpp/language/constexpr`.

constexpr-конструктори

- Якщо об'єкт має constexpr-конструктори, він може повертатися із constexpr-функцій чи присвоюватися constexpr-змінним.

```
struct Ratio{
    int num, den;
    constexpr Ratio(int x, int y): num(x), den(y){}
};

constexpr Ratio operator*(const Ratio& lhs, int rhs){
    return Ratio(lhs.num*rhs, lhs.den);
}

constexpr Ratio doublificy (const Ratio& x){
    return x*2;
}
```

Вимоги до constexpr-конструкторів

- Всі аргументи мають задовольняти LiteralType, не може мати try-блоків а сам клас не може мати віртуальних базових класів .
- В C++11 такий конструктор **може містити** лише те, що і constexpr-функція в C++11.
- ◇ **Крім того**, кожен базовий клас та всі не-статичні члени повинні бути ініціалізовані або безпосередньо при оголошенні, або в списку ініціалізації, при чому всі їх конструктори мають теж бути constexpr а аргументи ініціалізації — константними виразами.
- Для C++14 вимоги пом'якшилися і є тими ж, що і для consexpr-функцій.
- C++20 дозволив constexpr конструкторам ініціалізувати не всі не-статичні члени даних, однак, перш ніж їх читати, потрібно виконати присвоєння – через заборону UB для коду, що виконується під час компіляції.
- В C++14 constexpr-конструктори можуть існувати навіть для не-LiteralType класів — для константної ініціалізації (`http://en.cppreference.com/w/cpp/language/constant_initialization`).

constexpr-union

- Для об'єднань можна ініціалізувати лише один член.
- C++20 дозволив змінювати активний член:

```
union uni_t {
    int  i;
    double d;
};

constexpr auto fn() {
    uni_t u;
    u.i = 3;     // Рівно один член має бути ініціалізованим
    u.d = 4.0; // OK в C++20

    return u.d;
}

static_assert(fn() == 4.0 );
```

- Детальніше див. `http://en.cppreference.com/w/cpp/language/constexpr`.

- В C++11 constexpr-методи автоматично вважаються const. C++14 пом'якшив цю вимогу, дозволяючи використанням в constexpr-виразах і (constexpr) функцій, що змінюють стан об'єктів.
- Оператор noexcept завжди повертає true для константних виразів, тому його можна використовувати, щоб визначити, чи результат обчислення constexpr-функції є constexpr — константою часу компіляції. Наступний код:

```
int x = 5;
constexpr bool b2 = noexcept( cfactorial (5));
           bool b3 = noexcept( cfactorial (x));
cout << b2 << " - " << b3 << endl;
```

  виведе: `1 - 0`.

- Статичні константи-члени класів для не-інтегральних типів слід оголошувати як constexpr[5]:

```
struct SRatios1{
    static constexpr Ratio r12{1,2};
};
```

[5] Не повністю підтримується MSVC2015.

constexpr та невизначена поведінка

- Стандарт стверджує, що constexpr із невизначеною поведінкою повинен бути "ill-formed" – іншими словами, бути помилкою компіляції.
- За допомогою цього можна тестувати прояви доволі широкого класу UB. Звичайно, як завжди, не у всіх компіляторів воно веде себе однаково.
- Приклад:

```
constexpr int y1=std::numeric_limits<int>::max()+1;
```

GCC:

```
<path>/main.cpp:12:53: warning: integer overflow in expression of type 'int' resul
2147483648' [-Woverflow]
   12 |     constexpr int y1=std::numeric_limits<int>::max()+1;
      |                                          ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^~
<path>/main.cpp:12:54: error: overflow in constant expression [-fpermissive]
   12 |     constexpr int y1=std::numeric_limits<int>::max()+1;
```

MSVC 2017 скомпілював, правда:

```
<path>\main.cpp(12): warning C4307: '+': signed integral constant overflow
```

constexpr та невизначена поведінка

- Ще приклад:

```
const int g = 5;
int main() {
    // constexpr const int* p = nullptr; // Error
    constexpr const int* p = &g; // OK
    constexpr int y2 = *p;
}
```

- Подробиці – див. посилання, звідки приклад: "Exploring Undefined Behavior Using Constexpr".
- Цікавий погляд – constexpr як ще одна платформа, для якої компілюється код: `https://foonathan.net/2020/10/constexpr-platform/`. Код виконується компілятором.
- ◇ За допомогою `std::is_constant_evaluated()` можна визначити, чи функція виконується компілятором, дозволяючи надати іншу реалізацію для цього випадку – по аналогії із #ifdef для вибору runtime-платформ.

constexpr віртуальні функції

- C++20 дозволив constexpr функціям бути віртуальними.
- constexpr-віртуальна функція може заміщати не-constexpr і навпаки.

```
struct base_t{
    constexpr virtual ~base_t() = default;
    virtual int value() const = 0;     // non-constexpr
};
struct derived_1_t : base_t{
    constexpr int value() const override { return 1; }
};
struct derived_2_t : base_t{
    constexpr int value() const override { return 2; }
};
constexpr auto summarize(){
    const derived_1_t d1;
    const derived_2_t d2;
    const base_t* pb1 = &d1;
    const base_t* pb2 = &d2;

    return pb1->value() + pb2->value();
}
static_assert(summarize() == 1 + 2);
```

- ◇ GCC12 хоче `constexpr virtual ~derived_1_t() {}`, =default йому мало.

constexpr в C++26

- C++26 дозволив конструктори та деструктори (разом із іншими функціями) класів з віртуальним успадкуванням бути constexpr.
  - Це дозволяє зробити constexpr потоки iostream.
  - Тепер лише корутини не можуть бути constexpr-функціями.

constexpr dynamic_cast та typeid

- Разом із підтримкою віртуальних функцій, дозволено і dynamic_cast.
- ◇ typeid теж дозволено, але, оскільки, поки std::type_info не має constexpr членів, зробити із ним щось не вдасться.
- Приклад:

```
template<typename Base, typename Derived>
constexpr auto downcasted_get(){
    const Derived d;
    const Base& base = d;
    const auto& derived = dynamic_cast<const Derived&>(base);
    return derived.value();
}

static_assert(downcasted_get<base_t, derived_1_t>() == 1);
static_assert(downcasted_get<base_t, derived_2_t>() == 2);

// Помилка компіляції
// static_assert(downcasted_get<derived_1_t, derived_2_t>() == 2);
```

std::is_constant_evaluated()

- C++20 додав функцію std::is_constant_evaluated(), за допомогою якої можна перевірити, чи код виконується під час компіляції[6].
  - ◇ Тому є сенс її використовувати лише в constexpr-функціях.
  - ◇◇ Звичайні функції ніколи не constant evaluated, навіть якщо оптимізатор компілятор їх потім обчислив, а consteval – завжди константні.
  - ◇ Увага, у виразах, які обчислюються константно, типу static_assert(), "`if constexpr`", аргументах шаблонів, вказанні розмірів масивів, ініціалізації constexpr змінних тощо[7] ця функція завжди поверне true.
- У випадку з константним обчисленням, компілятор перевіряє, чи код так скомпілюється, і якщо ні – повертається до runtime-варіанту.
- Детальніше див. `https://gist.github.com/Som1Lse/5309b114accc086d24b842fd803ba9d2`.

[6] Формально, це твердження не зовсім вірне – мова про Manifestly constant-evaluated, див. is_constant_evaluated() та Manifestly constant-evaluated expressions, але суть передає.

[7] Список тут: `https://en.cppreference.com/w/cpp/language/constant_expression#Manifestly_constant-evaluated_expressions`.

std::is_constant_evaluated()

- Приклад:

```
constexpr int fn2(int x){
    if (std::is_constant_evaluated()){
        return x;
    }
    return x+1;
}

    .....
    int v = 7;
    constexpr auto r4 = fn2(7); // const evaluated
    cout << fn2(v) << endl;     // non-const evaluated
    cout << fn2(7) << endl;     // non-const evaluated

    // В першому випадку константне обчислення неможливе через v:
    const int a = std::is_constant_evaluated() ? v : 1;
                                        // non-const evaluated
    const int b = std::is_constant_evaluated() ? 2 : v;
                                        // const evaluated
```

Приклади **невірного** використання

- В if constexpr – завжди true, тому результат цієї ф-ції завжди x, ніколи не x+1:

```
constexpr int fn2c(int x){
    if constexpr (std::is_constant_evaluated()) {
        return x;
    }
    return x+1;
}
```

- Тут теж, тому перших два елементи tuple завжди будуть рівні 1:

```
constexpr auto fn3(int x){
    constexpr int v1 = std::is_constant_evaluated();
                                        // Завжди true, v1 == 1
    constexpr int v2 = std::array<int, std::is_constant_evaluated()>{}.size();
                                        // Завжди true, v2.size() == 1
    if (std::is_constant_evaluated()) {
        return std::make_tuple(v1, v2, x);
    } else {
        return std::make_tuple(v1, v2, x+1);
    }
}
```

constexpr try-catch

- Дозволено використовувати try-catch блоки в constexpr функціях.
- throw все ще заборонений – тому catch() просто ігнорується.
- Дозволяє писати функції, які потребують виключень в runtime варіантах.
  - Наприклад, із constexpr new.

constexpr asm-вставки

- Дозволено asm-вставки, що не виконуються під час компіляції:

```
constexpr auto add(const std::array<int, 4> &a,
                   const std::array<int, 4> &b){
    std::array<int, 4> res;
    if (std::is_constant_evaluated()){
        for(size_t i = 0; i<a.size(); ++i)
            res[i] = a[i] + b[i];
    } else {
        asm("    .....    "); // Optimized using SIMD
    }
    return res;
}
```

- ◇ MSVC не підтримує асм-вставки для x86-64.

Constexpr new – C++20

- Виділення динамічної пам'яті не дуже асоціюється із обчисленнями часу компіляції.
- Однак, багато коду, цілком придатного для виконання під час компіляції – наприклад, std::vector відомого розміру, використовує new.
- Тому в C++20 додали constexpr варіанти new, із наступними обмеженнями:
  - ◇ Щоб вираз був обчислений під час компіляції – щоб вважався core constant expression[8], пам'ять має бути звільненою до виходу із функції, де вона виділена.
  - ◇ Динамічна алокація може відбуватися або глобальним new, або викликом std::allocator<T>::allocate.
  - ◇ Аналогічні вимоги стосуються деалокації.
- Додатково було зроблено виняток для класів із нетривіальним constexpr деструктором, який звільняє всю пам'ять[9].
  - ◇ Це дозволяє створювати об'єкти типу vector чи string під час компіляції.

[8] https://en.cppreference.com/w/cpp/language/constant_expression

[9] https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0784r7.html

Тривіальний приклад

- Пам'ять звільняється до виходу із функції:

```
constexpr size_t parse_data(const char data[]){
    size_t str_sz = 0;
    while(data[str_sz] != 0)
        ++str_sz;
    size_t idx = 0;
    auto temp_buf = new char[str_sz];
    for(size_t i = 0; i<str_sz; ++i){
        if( data[i] >= '0' && data[i] <= '9' ) {
            temp_buf[idx++] = data[i];
        }
    }
    delete[] temp_buf;
    return idx;
}
```

- Перевірити можна так:

```
constexpr char text1[] = "abc123xyz";
static_assert(parse_data(text1) == 3, "Not evaluted to constant");
```

Якщо функція не буде обчислена під час компіляції – станеться помилка.

- При тому, оскільки UB під час компіляції заборонені, якщо в функції станеться memory leak, вихід за межі абощо, станеться помилка компіляції.
- ◇ Приклад повідомлення MSVC:

```
error C2131: expression did not evaluate to a constant
note: failure was caused by out of range index 13;
                            allowed range is 0 <= index < 10
.....................
error C2131: expression did not evaluate to a constant
note: failure was caused by allocated storage
                                          not being deallocated
```

- ◇ GCC та Clang теж дають цілком зрозумілі повідомлення.

```
template <typename T>
class VectorT {
    T *data_m;                  size_t size_m;
public:
    using value_type = T;
    constexpr VectorT(const T& val, size_t copies) noexcept : size_m(copies) {
        data_m = new T[copies];
        for(size_t i = 0; i<copies; ++i){           data_m[i] = val;        }
    }
    constexpr VectorT() noexcept : VectorT(T{}, 1){     }

    constexpr void swap(VectorT& rhs) noexcept {
        std::swap(rhs.size_m, size_m);          std::swap(rhs.data_m, data_m);
    }
    constexpr VectorT(const VectorT& rhs) noexcept : size_m(rhs.size_m){
        data_m = new T[rhs.size_m];
        for(size_t i = 0; i<rhs.size_m; ++i){   data_m[i] = rhs[i];         }
    }
    constexpr VectorT& operator=(const VectorT& rhs) noexcept {
        VectorT temp{rhs};          swap(temp);         return *this;
    }
    constexpr ~VectorT() noexcept {         delete[] data_m;    }

    constexpr size_t size() const noexcept { return size_m; }
    constexpr T&       operator[](size_t i)       noexcept { return data_m[i]; }
    constexpr const T& operator[](size_t i) const noexcept { return data_m[i]; }
```

- Користуючись константним контейнером із попереднього прикладу, можна робити всілякі фокуси:

```
template <typename T>
constexpr VectorT<T> concat(const VectorT<T>& lhs, const VectorT<T>& rhs ){
//constexpr VectorT<T> concat(VectorT<T> lhs, VectorT<T> rhs ){
    VectorT<T> res{T{}, lhs.size() + rhs.size()};
    size_t idx = 0;
    for( ; idx < lhs.size(); ++idx){ res[idx] = lhs[idx]; }
    for(size_t i=0 ; i < rhs.size(); ++i, ++idx){ res[idx] = rhs[i]; }
    return res;
}
constexpr auto fn2(){
    VectorT<int> v1{5, 22}, v2{7, 13};
    auto v3 = concat(v1, v2);
    return v3.size();
}
constexpr auto fn3(){
    VectorT<int> v1{2, 2}, v2{3, 3};
    auto v3 = concat(v1, v2);
    auto res1 = std::accumulate(v3.begin(), v3.end(), 0);
    return res1;
}
    .....
static_assert(fn2() == 35, "Not evaluted to constant");
static_assert(fn3() == 13, "Not evaluted to constant");
```

- Однак, безпосередньо скористатися даними, створеними під час компіляції, не вдасться.
- Для цього потрібно їх скопіювати у щось типу std::array[10]:

```
constexpr auto get_vectors(){
    VectorT<int> v1{2, 2}, v2{3, 3};
    auto s3 = concat(v1, v2);
    return s3;
}

constexpr auto get_array(){
    constexpr auto N = get_vectors().size();
    std::array<char, N> arr{};
    std::copy_n(get_vectors().begin(), N, std::begin(arr));
    return arr;
}

void test_str() {
    static_assert(!get_array().empty());
    // Не буде компілюватися -- містить дані, виділені під час компіляції
    // constexpr auto str = get_array();

    // OK -- дані в std::array<int, N>
    constexpr auto result = get_array();
}
```

- При чому, обчислювати дані доводиться двічі.

[10] Станом на червень 2022, MSVC 2019 не вміє працювати із цим кодом

- Ще приклад[11]:

```
constexpr std::string join(std::vector<std::string> vec, char delimiter) {
    std::string result = std::accumulate(
                std::next(vec.begin()), vec.end(),
                vec[0],
                [&delimiter](const std::string& a, const std::string& b) {
                                return a + delimiter + b;
                });
    return result;
}

    ..................
static_assert(join({"one", "two", "three"}, ';') == "one;two;three"sv);
```

[11] Взято тут: https://stackoverflow.com/questions/69498115/c20-constexpr-vector-and-string-not-working.

C++26

- C++26 запропонував вирішення давньої неузгодженості – тепер можна буде робити constexpr-посилання на локальні constexpr-змінні:

```
void fn(){
    constexpr int  i = 123;
    constexpr auto& ir = i;
}
```

◇ Проблема була в тому, що локальні змінні не мають константної, відомої на момент компіляції, адреси, навіть якщо вони constexpr. Для вирішення запропоновано ідею "Symbolic addressing" – нехай адреса буде локальною лише відносно кадру стеку.

- "Things C++26 define_static_array can't do" – есей про інструмент, який мав би зменшити використання "constexpr two-step", але, як показує автор, поки повністю не заміняє його.

constexpr – посилання

- https://habr.com/ru/post/579490/ – "Дизайн и эволюция constexpr в C++".
- https://www.cppstories.com/2021/constexpr-new-cpp20/
- https://oleksandrkvl.github.io/2021/04/02/cpp-20-overview.html#constexpr-features.
- https://quuxplusone.github.io/blog/2023/09/08/constexpr-string-firewall/ – чому не варто оголошувати std::string чи std::vector як constexpr змінні.
- https://biowpn.github.io/bioweapon/2024/02/17/constexpr-consteval-function.html

consteval (C++20)

- Constexpr функції виконуються в runtime для аргументів, для яких не вдалося обчислити під час компіляції. Іноді це незручно.
- Специфікатор consteval оголошує так звану immediate function, яка гарантовано буде обчислюватися під час компіляції.
- ◇ Більш строго, вимога до consteval-функцій стосується того, як інтерпретувати мову – стандарт нічого не каже про час компіляції чи час виконання, він лише про те, що така функція, за кожного виклику, має давати константний вираз[12].
- ◇ Якщо це неможливо – станеться помилка компіляції.

```
consteval int pow2(int n) {
    return n*n;
}
```

[12] Детальніше див. C++: constant expression.

- Викликати їх можна лише для константних сутностей.
- Тобто, такий код компілюється:

```
int r1 = pow2(100);
constexpr int a = 5;
int r2 = pow2(a);

const int b = 5;
int r3 = pow2(b);
```

◇ А такий – ні:

```
constexpr int calc(int n){
    return 2*pow2(n);
}
```

```
.....
int x = 5;
int r3 = pow2(x);
```

- Вимоги – ті ж, що і для constexpr функцій.
- Вказівники на такі функції можна отримувати, але вони не повинні consteval-контекст покидати.
- Детальніше див. `https://en.cppreference.com/w/cpp/language/consteval`.

- Через особливості стандарту[13], аргументи навіть consteval функцій не є constexpr.
- Тому такий код некоректний:

```
consteval bool isMatch(const std::string_view str) {
    if constexpr (str == "firstMatch"sv) {
        return true;
    } else if (str == "secondMatch"sv) {
        return true;
    }
    return false;
}
```

Потрібно прибрати constexpr – і результат буде якраз (неочікувано...) очікуваним, код буде обчислено під час компіляції.

- Внаслідок цього, повідомити про помилку static_assert() абощо не вдасться – але, як не дивно, можна кинути виключення – це спричинить помилку компіляції в потрібному місці, хай може і з дещо дивним повідомленням.
- Див. також обговорення "How to compare string_view using if-constexpt in a constexpr context", "How to implement constexpr string_view multiplication with a number"

[13] Зізнаюся, не до кінця зрозумілі автору цих презентацій. Див. також есей на тему, як це обмження (потворно) можна обійти: "std::nontype_t: What is it, and Why?".

constinit (C++20)

- static storage duration – змінні, що існують весь час роботи програми. Це глобальні та статичні змінні.
- Вони можуть ініціалізуватися або під час компіляції, або вже під час виконання, до першого використання[14].
  - З другим варіантом пов'язаний static initialization order fiasco – порядок їх конструювання не визначений, і якщо між ними є залежність – маємо проблему.
- consinit дозволяє гарантувати ініціалізацію під час компіляції.
  - Якщо це неможливо, буде помилка компіляції.
- constexpr-змінні є автоматично constinit.
- Однак, на загал, constinit змінні можна вільно змінювати.

[14] Наприклад, GCC використовує спеціальні функції-конструктори (не плутати із конструкторами класів), що викликаються перед main(). Приклад із "дикої природи" див. тут: `http://indrekis2.blogspot.com/2015/11/c-arm-gcc-stm32-coide.html`.

- Застосовувати constinit можна як до static storage duration-змінних, так і до thread_local-змінних.
- Додатково, для TLS-змінних таке оголошення повідомляє компілятору, що змінна вже ініціалізована і дозволяє не використовувати приховану синхронізацію:

```
extern thread_local constinit int x;
```

- Потребує наявності constexpr-конструктора, але не constexpr-деструктора – оскільки це змінна.
- Із constexpr, consteval, constinit одночасно дозволено використовувати лише один.

C++23

- C++23 дозволив static constexpr-змінні в constexpr-функціях:

```
constexpr char xdigit(int n) {
  static constexpr char digits[] = "0123456789abcdef";
  return digits[n];
}
```

- Приклад з `https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2647r1.html`.
- Вимоги дуже релаксовано – наприклад, можна викликати не constexpr-функції. Це зроблено, на випадок, якщо якісь функції будуть ставати constexpr в майбутньому. Подробиці див. "Relaxing some constexpr restrictions".

C++26

- Дозволено[15] перетворення static_cast в void* і з void*, якщо вказівник справді вказує на об'єкт відповідного типу в constexpr-функціях.
- Це відкриває шлях до constexpr std::any, std::format та constexpr-обгорток над callable.
- Детальніше див. документ з пропозицією: "constexpr cast from void*: towards constexpr type-erasure".
- Більшість стандартних контейнерів зроблено consexpr. Однак, std::hash (поки?) не consexpr і, щоб скористатися (як constexpr) контейнерами, які її потребують – std::unordered_XXX, потрібно надавати свою.
- Багато стандартних виключень зроблено constexpr.

[15] Точніше, поки – запропоновано так зробити.

Лямбда-функції

- C++11 додала лямбда-функції — можливість створювати анонімні функції безпосередньо біля місця їх використання.
- Дуже корисна річ при роботі із STL!
- Зазвичай, краще щоб лямбда-функції були маленькими — для великих і складних краще створити іменовану функцію.
- Приклад:

```
vector<int> v1 {1,4,2,7,3};

sort( v1.begin(), v1.end(), [](int x, int y){ return x>y;} );

for_each( v1.begin(), v1.end(), [](int x){ cout << x << ", "; } );
cout << endl;
```

виведе:

```
7, 4, 3, 2, 1,
```

Тип лямбда-функції:

- Можна сказати, при тому створюється функтор хитрого типу, із константним оператором operator(), який містить тіло лямбда-функції. Див. також слайд "Езотеро-auto" у відповідній презентації.
- Тобто, технічно,

```
sort( v1.begin(), v1.end(), [](int x, int y){ return x>y;} );
```

реалізовано якось так[16]:

```
class __lambda1{
public:
    bool operator()(int x, int y) const { return   x>y;}
};

sort( v1.begin(), v1.end(), __lambda1() );
```

Приклад вище — для GCC 5.2. Знати конкретний тип лямбда-функцій ніколи не потрібно! Він свідомо приховується.

- Різні лямбди мають різний тип.
- Такий тип називається **замиканням** (closure).

[16] Технічно оголошення б відрізнялося, але для ілюстрації зійде.

- Скориставшись auto, можна зберегти лямбда-функцію у змінну — фактично, дати їй ім'я і звертатися до неї потім.

```
auto f1 = [](){cout << "Hello!" << endl;};
f1 ();
```

На практиці такий фокус корисний не часто, хоча й іноді зручний. Не зловживайте ним!

- Наприклад, лямбда-функції можна використовувати як компаратори контейнерів, правда, не без певних ускладнень:

```
auto revComp = [](int a, int b){ return a > b; };
std :: set<int, decltype(revComp)> s(revComp);
```

Оскільки навіть "порожні" – без захоплення, замикання не можна створювати за замовчуванням (до C++20), код:

```
std :: set<int, decltype(revComp)> s;
```

не компілюватиметься, доводиться передавати їх явно.

Захоплення змінних

- Лямбда-функції доступні глобальні, статичні змінні та змінні просторів імен.
- Якщо їй потрібен доступ до локальних змінних, вона може їх **захопити**:

```
vector<int> v2;        .....................
int  minimal = 13;
auto n = count_if( v2.begin(),  v2.end(),
          [minimal](int  x){  return x>minimal; } )
```

- Замикання зберігає в собі такі захоплені змінні. Умовно:

```
class  __lambda2{
     int  minimal_m;
public:
     __lambda2(int minimal): minimal_m(minimal){}
     bool operator()(int  x)  const { return x>minimal_m; }
};

sort( v1.begin(),  v1.end(),  __lambda1() );
```

- operator() за замовчуванням оголошується константним, тому лямбда-функція не може змінювати захоплені значення.
- Якщо це для чогось потрібно (таке буває рідко), її можна оголосити mutable:

```
[sm](int x) mutable { sm = 2*x; cout << sm << ", "; }
```

- Захоплення локальних змінних може відбуватися за значенням, як показано вище, або за посиланням — якщо лямбда-функція повинна модифікувати відповідний об'єкт[17], його не можна копіювати або це неефективно:

```
ofstream log_file("log.dat");
for_each( v1.begin(), v1.end(),
    [&log_file](int x) { log_file << x << ", "; } );
log_file << endl;
```

- Лямбда-функція (замикання) може прожити довше, ніж код, який її створив, тому, захоплюючи змінні за посиланням, слід пильнувати, щоб вони проіснували достатньо довго!

[17] Не плутайте із модифікацією членів класу-замикання!

- Квадратні дужки із списком захоплення називають lambda introducer[18].
- Немає можливості зробити захоплення змінних за константним посиланням.
- ◇ Звичайно, константні змінні буде захоплено як константне посилання.
- До C++14 не було безпосереднього способу захопити об'єкти, які можна лише переміщати.
- Захоплювати можна кілька змінних зразу:

```
vector<int> v2;        .....................
int  minimal = 13;
int  maximal = 17;
auto n = count_if( v2.begin(),  v2.end(),
          [minimal,  maximal](int  x){  return  x>minimal && x<maximal; } );
```

- Можна комбінувати захоплення по значенню і за посиланням:

```
for_each( v2.begin(),  v2.end(),
   [minimal,  maximal, &log_file]( int  x){
      if  (x>minimal && x<maximal) { log_file << x << ", "; }
   } );
```

[18] Цікавинка: Якщо для шаблонів два ">" підряд дозволили, з лямбда-introducer так вирішили не робити, тому код arr[[]()return 1;()] не компілюватиметься, навіть пробіли не рятують. Див. https://shafik.github.io/c++/2022/09/20/init-lambdas-array-ohmy.html

- Є можливість сказати: "захоплюй всі необхідні тобі змінні".
  - Нічого зайвого при тому не захоплюватиметься — лише реально використані в тілі лямбда-функції змінні потраплять у замикання.
  - Однак, на загал, це погана практика — лямбда-функції призначені бути маленькими і компактними, навіщо їм велика кількість без розбору захоплених змінних?
- Захоплювати всі по значенню:

```
auto n = count_if( v2.begin(), v2.end(),
          [=](int x){ return x>minimal && x<maximal; } );
```

- Захоплювати всі за посиланням:

```
for_each( v2.begin(), v2.end(),
  [&](int x){
    if (x>minimal && x<maximal) { log_file << x << ", "; }
  } );
```

- Вказавши спосіб захоплення за замовчуванням, можна уточнити, як захоплювати конкретну змінну:

```
for_each( v2.begin(), v2.end(),
  [=, &log_file](int x){
    if (x>minimal && x<maximal) { log_file << x << ", "; }
  } );
```

Доступ до членів класу

- Лямбда-функція в методі класу може мати доступ до його членів, якщо захопить this:

```
class C{
    int m = 1;
public:
    size_t fn(const vector<int>& v){
        return count_if( v.begin(), v.end(),
                         [this](int x){ return x>m; } );
    }
};
```

- Фактично, члени класу завжди захоплюються за посиланням.
- Додатково можна захоплювати інші локальні змінні.

- Страуструп стверджує, що this можна використовувати після "[&]", але не можна після ["="], хоч це і не має особливого сенсу — його і так захоплює в такій ситуації. Однак, і GCC і MSVC успішно компілюють такий код в методі вище:

```
auto test1 = [=,this]( int x){ return x>m; };
auto test2 = [&,this]( int x){ return x>m; };
```

  - ◇ C++20 впорядкував це трохи – дозволив обидва варіанти.
- Також стверджується, що після "=" можна захоплювати лише за посиланням, і навпаки. Однак, GCC скомпілював наступне. MSVC поскаржився.

```
int k, l;
auto test3 = [&,k](int x){ return x>m; };
auto test4 = [=,k](int x){ return x>m; };
```

- Не зловживайте багами компілятора! :-)

Додатково про лямбда-функції

- Якщо функція не має аргументів, порожні круглі дужки можна опускати:

```
thread t1{ []{ cout << '.'; cout << endl;} };
t1.join();
```

- Можна захоплювати va...-аргументи шаблонів із змінною кількістю аргументів.
- Лямбда-функція може містити практично все те, що і звичайна функція. Однак, для великих функцій, краще писати їх звичним чином — великі лямбди важко читати і підтримувати.
- Подробиці про об'єкт-замикання можна почитати тут: `http://en.cppreference.com/w/cpp/language/lambda`.
- Виклики лямбда-функцій можна використовувати при вказанні аргументів за замовчуванням, але при тому вони не можуть нічого захоплювати.

```
int fx(int x = [](){ return 1;}() ){
    return x;
}
int zz = fx();
```

Додатково про лямбда-функції

- Якщо лямбда-функція нічого не захоплює, її можна перетворювати[19] у вказівник на функцію.
- Цікавий курйоз, пов'язаний із даною можливістю – функція, яка приймає лише вказівники на функції, сприйме лямбду, якщо виконати розіменування:

```
template<class F>
void call_twice(F *f) {      (*f)();      (*f)();      }
auto two = []() {
    puts("two");
};
call_twice(two); //! Error
call_twice(*two); //! OK
```

Причина – оператор * змушує здійснити приведення до вказівника на функцію та його розіменування. Але потім функція "охоче" розпадається (decay) до вказівника на функцію. Тому таке – теж OK:

```
call_twice(********two); //! OK
```

Приклад взято тут: `https://quuxplusone.github.io/blog/2020/03/31/asterisk-and-ampersand/`.

[19] Існує відповідний оператор перетворення у класі лямбда-функції. Подробиці для MSVC див.

Тип, що повертається

- Якщо лямбда-функція має рівно один return, тип результату виводиться автоматично. (Якщо не має жодного, вважається що це void).
- Якщо є більше одного return — в C++11 це помилка, тип результату слід вказувати явно:

```
auto t2 = [](int x, int y) -> int
          { if (x>y) return 1; else return -1; };
```

- В C++14 лямбда-функція, із кількома return і без явного вказання типу, поводиться так, як звичайна функція із типом результату auto — тип виводиться, якщо результат для різних return співпадає, все OK, інакше — помилка компіляції, тому код вище можна записати і так:

```
auto t2 = [](int x, int y)
          { if (x>y) return 1; else return -1; };
```

В принципі, GCC 4.9, 5.2, MSVC компілюють цей код і без вказання C++14[20].

[20] Взагалі, для MSVC його вказати і не можна — студія використовує те, що вже реалізовано, але макрос __cplusplus не виправляє. Вказання стандартів Студія почала підтримувати із 17-го (року, версії, стандарту).

Immediately-invoked function expression — IIFE

- Ініціалізація змінної може потребувати складної логіки.
- Якщо це константа – то доведеться писати все в один рядок, користуючись або сторонньою функцією (яка не матиме необхідного контексту – і їй його доведеться передавати), або складними, вкладеними тернарними операціями. І одне рішення і друге – субоптимальні.
- Приклад:

```
const int var = [&] {
    if (cond) // Оголошено трохи вище.
        switch(mode){
            case N_MODE: return 1;
                .....
        }
    else
        return def_val;
}(); // Виклик відбувається тут!
```

- Такий код трохи неочевидний. Основна проблема – не видно, що відбувається виклик функції.
- Тут: `https://www.bfilipek.com/2016/11/iife-for-complex-initialization.html` пропонується декілька покращень читабельності.
- Даємо тій лямбда-функції ім'я, тоді викликаємо:

```
auto initialiser = [&] { ..... };
const int var = initialiser ();
```

- Або – виклик за допомогою invoke (C++17):

```
const int var = std::invoke([&] { ..... };
```

Лямбда-функції в C++14

- C++14 підтримує авто-виведення типів аргументів лямбда-функцій — такі собі лямбди-шаблони.
- Називаються такі лямбда-функції узагальненими – generic lambdas:

```
for_each( v1.begin(), v1.end(), [](auto x){ cout << x << ", "; } );
```

- Можна задавати аргументи за замовчуванням:

```
auto lam1 = [](int x, int y = 0){
                .....
    };
lam1(3);
lam1(4,5);
```

Лямбда-функції в C++14

- З'явилася можливість задавати ініціалізатори в списку захоплення:

  //! Захоплення із переміщенням —— зокрема, для *move—only* типів*:*
  [x=move(x)] { f(x); }

  //! Можна навіть переіменовувати*:*
  [y=move(x), z = k, s = a + b] { f(y, z, s); }

  //! Приклад з *http://en.cppreference.com/w/cpp/language/lambda*

```
int x = 4;
auto y = [&r = x, x = x + 1]()
    {
        r += 2;
        return x + 2;
    }(); // x = 6, y = 7 after the call.
```

- При тому створюється auto-змінна, яка ініціалізується згідно переданого ініціалізатора.
- Це дозволяє захоплювати змінні з переміщенням.

Лямбда-функції в C++17

- C++17 додав constexpr-лямбда-функції. Правда, якщо це ключове слово не вказати, а функція вимогам до constexpr задовольняє – вона буде constexpr автоматично:

```
// Constexpr lambda -- explicit
auto z1 = [](int x) constexpr { return x + 5; }(11);

int z2 = 42;
auto answer = [z2](int x) constexpr { return z2 + x; }(11);

// Constexpr lambda -- implicit
constexpr int z3 = 42;
constexpr auto cel = [z3](int x) { return z3 + x; };
static_assert(cel(10) == 52);
```

Лямбда-функції в C++17

- Іноді потрібно захопити об'єкт, на який вказує this, а не сам вказівник – наприклад, якщо об'єкт житиме менше, ніж лямбда-функція.
  - В C++ 14 це можна зробити так: `[self = *this](){ self.method(); }`.
  - В C++ 17 – прямолінійніше: `[*this](){ method(); }`.
    - Для такого використання метод method() має бути константним.
    - Для неконстантних звертань лямбда-функція має бути mutable.

Лямбда-функції в C++20

- Хоча в C++14 лямбда-функції можуть бути узагальненими: `[](auto a){...}`, поводячись схоже на шаблони, особливого контролю над цим механізмом не було, а для доступу до справжніх типів аргументів потрібно було застосовувати decltype.
- C++20 додав безпосередні шаблонні лямбда-функції:

```
[]<typename T>(std::vector<T>& a, const auto& b) {
    a.push_back(b);
};

[]<typename T>(std::vector<T>& a, T&& b) {
    a.push_back(std::forward<T>(b));
};

[]<typename T, typename U>(const T& a, const U& b) {
    return a + b;
};
```

- Для таких лямбда-функції operator() замикання стає шаблоном.

- Variadic templates:

```
[]<typename... Ts>(auto fn, Ts&&... args){
    return fn(std::forward<Ts>(args)...);
};
[](auto fn, auto&&... args){ // В C++14
    return fn(std::forward<decltype(args)>(args)...);
};
```

- Краще, звичайно, так:

```
auto tmpl_lambda_fwd2 = []<typename T, typename... Ts>(T&& fn, Ts&&... args){
    return std::invoke(std::forward<T>(fn), std::forward<Ts>(args)...);
};
```

◇ Тепер воно прозоро дружить із вказівниками на члени, але – з перевантаженням все ще не дружить. Якщо дуже хотіти, можна так, але це – поганий спосіб:

```
using pb_ptr = void(std::vector<std::string>::*)(const std::string &);
tmpl_lambda_fwd2(static_cast<pb_ptr>(&std::vector<std::string>::push_back),
                 std::ref(sv2), "klm"s);
```

- Також можна використовувати концепції:

```
auto tmpl_lambda_fwd3 = []<typename T, copy_constructible... Ts>
        (T&& fn, Ts&&... args)
         requires std::invocable<T, Ts...>
{
    return std::invoke(std::forward<T>(fn), std::forward<Ts>(args)...);
};
```

Lambdas in unevaluated contexts

- До C++20 лямба-функції не можна було використовувати у випадках, де не відбувається обчислень – як аргументи decltype(), sizeof(), typeid(), noexcept(), концепції та обмежень тощо.
- ◇ Причина – уникнення просочування сигнатур лямбд в сигнатури і проблем з заплутуванням імен: "Wording for lambdas in unevaluated contexts" (2017).
- ◇ Причина причини – дві однакові лямбда-функції мають різні типи:

```
auto a = [](){}; // типи a і b — різні
auto b = [](){};
```

В стандарт додали виняток із цього правила для сигнатур функцій, об'єктів тощо.

Default constructible and assignable stateless lambdas

- Тепер можна робити так:

```
std::set<std::string, decltype(
                    [](const auto& lha, const auto& rha)
                        { return lha > rha; })> s1;
```

Раніше доводилося:

```
auto cmpl = [](const auto& lha, const auto& rha)
                        { return l > r; };
std::set<std::string, decltype(cmpl)> s1{cmpl};
```

- Приклад використовує ще одне важливе покращення лямбд в C++20 – stateless lambda – лямбди, що нічого не захоплюють, мають конструктори за замовчуванням та оператор присвоєння.
- Завдяки цьому створюється об'єкт-компаратор – інакше довелося б вручну, а також працює наступний код:

```
auto s1as = s1; // Requires default-assignable lambdas
```

- `https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0624r2.pdf` – відповідна пропозиція.

- Ще приклад із оригінальної пропозиції:

```
template <typename ...T, typename Predicate>
auto tsort(std::tuple<T...> const& tuple, Predicate const& pred) {
        ..................
}

auto tuple = std::make_tuple(std::array<int, 5>{}, 1, '2', 3.3);
tsort(tuple, [](auto const& a, auto const& b) {
    return std::integral_constant<bool, sizeof(a) < sizeof(b)>{};
});
```

Pack expansion in lambda init-capture

- До C++20 захоплювати у лямбду можна було по значенню або за посиланням.
- Якщо потрібно було перемістити окремі захоплені елементи, доводилося користуватися засобами C++14: `[arg = std::move(arg)](){.....}`.
- Якщо багато – запаковувати їх в кортеж і переміщати так або робити ще якісь трюки. Приклад – C++17:

```
template <class... Args>
auto delay_invoke_foo(Args... args) {
    return [tup=std::make_tuple(std::move(args)...)]() -> decltype(auto) {
        return std::apply([]( auto const&... args) -> decltype(auto) {
            return foo(args...);
        }, tup);
    };
}
```

- В C++20 додано пакети аргументів для захоплення у лямбдах:

```
template <class... Args>
auto delay_invoke_foo(Args... args) {
    return [... args=std::move(args)]() -> decltype(auto) {
        return foo(args...);
    };
}
```

- Приклад із оригінальної пропозиції:
`https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0780r2.html`.

Знову захоплення this – C++20

- Навіть якщо використати [=], до C++20 this захоплювався за посиланням.
- C++17 додав можливість явного захоплення this по значенню.
- C++20 – явне захоплення this за посиланням.
- Приклад:

```
struct S{
    void fn(){
        [=]{};          // captures this by reference, deprecated since C++20
        [=, *this]{};   // OK since C++17, captures copy of enclosing S
                        // in other words, capture this by balue
        [=, this]{};    // OK since C++20, captures this by reference --
                        // works with enclosing object as S&,
                        // same as [=] before.
    }
};
```

- Ремарка: видно, що хоча синтаксично this – вказівник, це legacy, фактично із об'єктом працюють як з посиланням.
- Лямбда-функції в ініціалізації членів класу, за потреби, захоплюють this автоматично:

```
struct S{
    int x = 1;
    int y{ [&]{ return x + 1; }() };  // Має доступ до x -- мусила захопити this
};
```

Захоплення structutred binding

- Тепер можна і так:

```
auto tlsb = [](){ return make_tuple(1, 2.34, "xyz"s); };
auto [tl1a, tl2a, tl3a] = tlsb();

auto sb_lmbd = [tl2a](){ auto x = tl2a; };
```

- До C++20 така можливість не підтримувалася.
- "Destructuring Lambda Expression Parameters" – есей на суміжну тему.

Посилання

- https://habr.com/ru/post/548938/ – альтернатива, яка type erasure, щоб могти лямбда-функції передавати за вказівником і взагалі використовувати замість std::function.
- https://en.cppreference.com/w/cpp/language/lambda
- https://leanpub.com/cpplambda – C++ lambda story, Bartlomiej Filipek.
- https://www.cppstories.com/2020/07/lambdas5ex.html/ – "5 Curious C++ Lambda Examples: Recursion, constexpr, Containers and More".
- "Closures?" – дещо закручений приклад втрат на використання замикань.

std::function

- Шаблон std::function служить узагальненим засобом зберігати та передавати Callable. Іноді їх ще називають polymorphic wrappers.
- Може зберігати будь-які Callable — будь-що, до чого можна звернутися із оператором виклику функції — (). Це можуть бути функції, лямбда-функції, результати bind(), та інші функтори.
- Приклад:

```
void fn1(int x) { cout << x << endl;};
std :: function<void(int)> fo1 = fn1;
fo1(5);
```

- Фактично, є узагальненням поняття вказівника на функцію.
- Зберігати функцію, користуючись auto, майже завжди швидше! Компілятор не часто може в тій же мірі оптимізувати function<>. Однак, далеко не всюди auto можливе.

auto vs function<>

- В оголошенні функції auto не застосуєш а function<> — можна:

```
//void decorator(auto fn) {
//    cout << "Started" << endl;
//    fn();
//    cout << "Stopped" << endl;
//}                                   // !ERROR!
```

```
void decorator(function<void()> fn) {
    cout << "Started" << endl;
    fn();
    cout << "Stopped" << endl;
}
```

- Можна також створити шаблон:

```
template <typename FT>    void decoratorT(FT fn) {
    cout << "Started" << endl;
    fn();
    cout << "Stopped" << endl;
}
```

Він буде трішки швидшим, але генеруватиме НОВІ функції для кожного інстанціювання — виклику із іншою функцією

- Також не можна використовувати auto, оголошуючи член класу який має бути Callable і не завжди можна це зробити для результату функції.

```
//! Error!
//auto get_cl1() -> decltype( []{} ){
//     return  []{cout << "ABC";};
//}

function<void()> get_cl2(){
    return  []{cout << "ABC";};
}

struct  CaStore{
    // auto   m_fn; //<ERROR!
    function<void()> m_fn;
};
```

- Шаблони часто дозволяють досягнути того ж результату, часто даючи невеликий приріст швидкодії, ціною роздування коду.

Приклади:

- **void** fn1(**int** x) { cout << x << endl;};

```
std :: function<void(int)> fo1 = fn1;
std :: function<void(int)> fo1a{fn1};
std :: function<auto(int)->void> fo1b{fn1};
fo1(5);
fo1a(5);
fo1b(5);
```

int fn2(**double** a , **int** b){ **return** a/b;};

```
std :: function<int(double, int)> fo2 = fn2;
cout << fo2(8.14,5) << endl;
```

- **struct** Functor1{
  void operator()()**const**{ cout << ".."; }
  };

```
std :: function<void(void)> fo3 = Functor1{};
fo3();
```

- ```
  class Functor2{
      int x_m;
  public:
      Functor2(int x) : x_m{x} {}
      bool operator()(int y)const{ return x_m>y; }
  };

      std::function<bool(int)> fo4 = Functor2{33};
      cout << boolalpha << fo4(5) << endl;
  ```

- .

  ```
  template <class T>
  class Functor3{
      T x_m;
  public:
      Functor3(T x) : x_m{x} {}
      bool operator()(T y)const{ return x_m>y; }
  };

      std::function<bool(float)> fo5 = Functor3<float>(33);
      cout << fo5(3.33) << endl;
      std::function<int(int)> fo5a = Functor3<float>(33);
      cout << fo5a(3.33) << endl;
  ```
  ```

- **template** <**class** T>
  class Functor4{
      T x_m;
  public:
      Functor4(T x) : x_m{x} {}
      template<**class** Y>
          bool operator()(Y y)**const**{ **return** x_m>y; }
  };

  ```
  std :: function<bool(double)> fo6 = Functor4<char>(44);
  cout << fo6(5) << endl;
  ```

- Лямбда-функції напряму рекурсивними бути не можуть, але можна так:

  ```
  std :: function<unsigned(unsigned)> factl
      = [&factl](unsigned x)
                  { return x<=1 ? 1 : (factl(x-1)*x); };
  cout << factl(5) << endl;
  ```

- function<> працює і з вказівниками на члени:

```
class Class1{
    int x_m;
public:
    Class1(int x) : x_m{x} {}
    bool test_value(int y) const { return x_m>y; }
    bool always_fail(int) const { return false; }
};

    std::function<bool(const Class1&, int)> fo7;
    Class1 c1(7);
    fo7 = &Class1::test_value;
    cout << fo7(c1, 5) << endl;
    fo7 = &Class1::always_fail;
    cout << fo7(c1, 5) << endl;
```

Подробиці function<>

- Має конструктор за замовчуванням, тому може бути не пов'язаним із жодним Callable. При спробі викликати такий об'єкт function<>, кине виключення std::bad_function_call.
- Має оператор присвоєння та метод swap().
- Приводиться до bool, повертаючи true, якщо має цільову функцію чи інший Callable, false, якщо ні.
- Поле target_type типу type_info з інформацією про тип Callable і шаблонний метод target<T>(), який повертає вказівник на нього, якщо target_type() == typeid(T) і nullptr в іншому випадку.
- Метод assign()[21] дозволяє вибрати нову ціль для function<>.
- Детальніше див. `http://en.cppreference.com/w/cpp/utility/functional/function`.
- Аналіз особливостей реалізації, "Inside std::function": `https://devblogs.microsoft.com/oldnewthing/20200513-00/?p=103745`, `https://devblogs.microsoft.com/oldnewthing/20200514-00/?p=103749`, `https://devblogs.microsoft.com/oldnewthing/20200515-00/?p=103755` (описано аспекти, не пов'язані із форвардингом аргументів).

[21] Визнано застарілим в C++17.

Інші варіанти збереження callable

- Щодо компромісів та рішень, зроблених під час розробки std::function, див. `https://quuxplusone.github.io/blog/2019/03/27/design-space-for-std-function/`. Нижче – альтернативні реалізації, за посиланнями із статті.
- `function_ref` – non-owning аналог, див. `http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0792r0.html`, `https://vittorioromeo.info/index/blog/passing_functions_to_functions.html`, `https://foonathan.net/blog/2017/01/20/function-ref-implementation.html`.
- `unique_function` — "a move-only std::function", `http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0228r3.html`.
- `inplace_function` – із явним керуванням буфером для SBO і вирівнювання: `https://github.com/WG21-SG14/SG14/blob/master/SG14/inplace_function.h`.
- `https://biowpn.github.io/bioweapon/2024/01/18/what-the-func.html` – есей про потребу в: std::move_only_function, std::copyable_function, std::function_ref, які, разом з std::function, всі є в C++26.

Додаткові обгортки callable в C++23 та C++26

- Для вирішення деяких проблем з std::function[22] в C++23 та C++26 було додано кілька варіацій:
  - ◇ std::move_only_function (C++23), розуміє const і noexcept
  - ◇ std::copyable_function (C++26), розуміє const і noexcept,
  - ◇ function_ref (C++26) – клас без володіння.
- Приклад:

```
std :: function_ref<bool(int) const noexcept>
std :: move_only_function<bool(int) noexcept>
std :: copyable_function<bool(int) const>
```

- Детальніше див. "Function wrappers".

[22] Не вміє працювати лише з movable-об'єктами, не знає про noexcept, ігнорує const, а також – доволі складний для оптимізації тип.

std::identity – C++20

- Оператор виклику об'єкту типу std::identity повертає свій аргумент без змін.

std::bind() — універсальний зв'язувач

- std::bind() дозволяє створити callable-об'єкт (який можна зберігати в function<>) із іншого (наприклад, функції), де частині аргументів задано фіксовані значення.
- Приклад:

```
auto b1 = bind(fn1, placeholders::_1, 125);
b1(3);                  // First arg: 3, second arg: 125

auto b2 = bind(fn1, 125, placeholders::_1);
b2(3);                  // First arg: 125, second arg: 3

auto bmixed = bind(fn1, placeholders::_2, placeholders::_1);
bmixed(3, 125);    // First arg: 125, second arg: 3

auto buseless = bind(fn1, placeholders::_1, placeholders::_2);
buseless(3, 125); // First arg: 3, second arg: 125
```

- placeholders::_X вказує використати той аргумент, який у виклику результату bind стоїть на місці X. Нумерація — від 1. Їх часто використовують із using-директивою...
- Кількість таких аргументів — на розсуд реалізації[23].

[23] В GCC 4.9-11.2, об'єкти-"заповнювачі" є по placeholders::_29, MSVC 2015-2020 — по placeholders::_20.

- Використовувати всі аргументи виклику не обов'язково:

```
auto b4 = bind(fn2, _2, _1, 42);
b4(1,2,3);          // Arg 1: 2, arg 2: 1, arg 3: 42
b4(1,2,3,7,9);      // Arg 1: 2, arg 2: 1, arg 3: 42
```

- Можна використовувати вкладені bind(). Вони використовуватимуть спільну нумерацію для placeholder-ів:

```
auto b5 = bind(fn2, _2, bind(fn1, _1, _2), _1);
b5(55,77);
```

Дасть:

```
First arg: 55, second arg: 77
Arg 1: 77, arg 2: 0, arg 3: 55
```

- Працює із вказівниками на члени:

```
struct Class1 {
    int x_m;
    Class1(int x) : x_m{x} {}
    bool test_value(int y) const{
        cout << "We have: " << x_m << ", comparing with: " << y <<
        return x_m>y;
    }
};
    Class1 obj1(13);
    auto b6 = std::bind(&Class1::test_value, &obj1, _1);
    b6(22);
```

дасть:

```
We have: 13, comparing with: 22
```

- Та з вказівниками на члени даних:

```
auto b7 = std::bind(&Class1::x_m, _1);
cout << b7(obj1) << endl;
```

- Можна також так:

```
cout << b7(&obj1) << endl;
cout << b7(make_shared<Class1>(31)) << endl;
```

- Типове використання (взято тут: `http://en.cppreference.com/w/cpp/utility/functional/bind`) — прив'язка генератора випадкових чисел до розподілу:

```
std::default_random_engine e;
std::uniform_int_distribution<> d(0, 10);
std::function<int()> rnd = std::bind(d, e); // зберігає копію e

std::cout << rnd();
```

- Технічна подробиця: тип результату bind() – на розсуд компілятора, але перевірити, чи callable є результатом bind() можна за допомогою `std::is_bind_expression<T>::value`.
- Використовує perfect forwarding.
- Детальніше про bind() див. `http://en.cppreference.com/w/cpp/utility/functional/bind` та `http://en.cppreference.com/w/cpp/utility/functional/placeholders`.

Підсумок bind()

- Завдяки йому bind1st і bind2nd застаріли, а ptr_fun, mem_fun, mem_fun_ref із C++98 — непотрібні, та й std::mem_fn із C++11 – не особливо.
- Зазвичай використання лямбда-функцій зручніше (див. приклади у l21_vector).
- Результат bind() можна зберігати в function<>.
- Станом на C++14 немає нічого, що не можна зробити без bind(), тому його використання не те щоб не рекомендується, але й не стимулюється.
- В C++11 лямбда-функції не могли захоплювати із переміщенням (`std::bind(f, 42, _1, std::move(v))`), вирази теж не захоплювалися (`std::bind(f, 42, _1, a + b)`, не працювали із перевантаженнями типів аргументів).
- При тому bind() ускладнює оптимізацію (важче інлайнити) та дозволяє викликати отриманий об'єкт із завеликою кількістю аргументів.
- Детальніше див. обговорення тут (єдине, разом із коментами – бо є помилки): `https://stackoverflow.com/questions/17363003/why-use-stdbind-over-lambdas-in-c14`.
- Подробиці реалізації: `http://blog.think-async.com/2010/04/bind-illustrated.html` –

C++20+

- C++20 додав `bind_front( F&& f, Args&&... args );`, який дозволяє частково застосовувати перші аргументи функції, без можливості перевпорядкування:

```
int fn3(int a, int b, int c);
    ..............
auto b8_1 = std::bind(fn3, 3, 125, std :: placeholders :: _1); // C++11
b8_1(35);
auto b8_2 = std::bind_front(fn3, 3, 125); // C++20
b8_2(35);
```

- C++23 додав `bind_back( F&& f, Args&&... args );` – те ж, але для останніх аргументів.
- C++26 планує додати варіанти цих функцій з константними callable:

```
#if __cpp_lib_bind_front >= 202306L
auto b8_3 = std::bind_front<fn3>(3, 125);
b8_3(35);
#endif
```

  Такі варіанти можуть бути ефективнішими.
- Детальніше див. `https://en.cppreference.com/w/cpp/utility/functional/bind_front`.

std::mem_fn()[24]

- Цей адаптер дозволяє зберегти вказівник на член так, ніби це звичайна функція, першим аргументом якої є об'єкт, для якого даний член слід викликати.
  - ◇ Mem – скорочення від member, не від memory.
- Приклад 1:

```
auto ss = mem_fn(&string::size);
string str_obj("xxx");
cout << ss( str_obj ) << endl;
```

- Приклад 2 — заповнюємо string_sizes розмірами стрічок із vs :

```
vector<string> vs = {"abc", "eeeee", "aa", "xxx"};
vector<size_t> string_sizes;
transform(vs.begin(), vs.end(), back_inserter( string_sizes ),
          mem_fn(&string::size));
```

Це ж, з допомогою лямбда-функцій:

```
transform(vs.begin(), vs.end(), back_inserter( string_sizes ),
          []( const string& s){return s.size();});
```

чи bind:

```
transform(vs.begin(), vs.end(), back_inserter( string_sizes ),
          bind(&string::size, _1));
```

- Приклад 3:

```
class Class2{
    int x_m;
public:
    Class2(int x) : x_m{x} {}
    void print_me1()const{ cout << "I'm: " << x_m << endl; }
    void print_me2()const{ cout << "Just " << x_m << endl; }
};

vector<Class2> c2a={3,7,11,22,5};
auto pm = mem_fn(&Class2::print_me1);
pm(c2a[0]);

for_each(c2a.begin(), c2a.end(), mem_fn(&Class2::print_me1));
for_each(c2a.begin(), c2a.end(), mem_fn(&Class2::print_me2));
```

- Це ж, з допомогою лямбда-функцій:

```
for_each(c2a.begin(), c2a.end(), [](const Class2& a){ a.print_me1();});
for_each(c2a.begin(), c2a.end(), [](const Class2& a){ a.print_me2();});
```

- Чи bind():

```
for_each(c2a.begin(), c2a.end(), bind(&Class2::print_me1, _1));
for_each(c2a.begin(), c2a.end(), bind(&Class2::print_me2, _1));
```

std::ref() та std::cref()

- std::ref<T>() та std::cref<t>()[25] повертають std::reference_wrapper<T> та std::reference_wrapper<const T>, відповідно.
- std::reference_wrapper<> — обгортка для (збереження) посилання.
- ref() та cref() використовуються там, де слід, щоб виведення аргументів шаблонів давало результат, який є посиланням (див. також розмову про виведення типів у розділі про auto і `http://en.cppreference.com/w/cpp/language/template_argument_deduction`).
- Приклади 1 і 2:

```
int a=11, b=12;
auto fr1 = bind(fn3, ref(a), ref(b), 3);
fr1();

unique_ptr<Class1> uniptr{new Class1(31)};
auto b7a = std::bind(&Class1::x_m, ref(uniptr));
cout << b7a() << endl;
```

[25] Чудово автоматично виводять типи, <T> вказано, щоб потім T можна було використати.

- Приклад 3:

```
void worker(int idt, std::mutex& m){....}

std::thread trd1( worker, 1, std::ref(m) );
```

- reference_wrapper<> може бути цінним і сам по собі, для збереження в контейнерах посилань — безпосередньо це неможливо, а іноді може бути корисним:

```
std:: list <int> l(10)={....};

std::vector<std::reference_wrapper<int>> v(l.begin(), l.end());
// shuffle потребує ітератор довільного доступу, тому
// його не можна використовувати із list. Але і копіювати
// елементи не хочеться ....
std:: shuffle (v.begin(), v.end(), std::mt19937{std::random_device{}()})
```

- Детальніше див. `http://en.cppreference.com/w/cpp/utility/functional/ref` та `http://en.cppreference.com/w/cpp/utility/functional/reference_wrapper` — звідси взято приклад вище.
- "What is std::ref?" – легенький есей на тему.

- Інші компоненти хедера <functional>, включаючи застарілі, розглядаються в розмові про STL.

std::invoke – C++17

- В C++ – 4 типи callable: функції, лямбда-функції, об'єкти-функції та результати роботи std::bind().
- Крім цих callable, є ще вказівники на члени.
- За потреби їх викликати уніфіковано, виникає проблема:
  - callable викликаються так: "fn(args);",
  - функції, на які вказує вказівник на член, за вказівником на об'єкт, викликають: "(arg1->*fn)(arg2,...);",
  - а для об'єкту: "(arg1.*fn)(arg2,...);".
- std::invoke уніфікує ці виклики.

```
struct Foo {
    Foo(int num) : num_(num) {}
    void print_add(int i) const { ... }
    int num_;
};
void print_num(int i) { ..... }
struct PrintNum {
    void operator()(int i) const { ..... }
};

    std::invoke(print_num, -9);              // invoke a free function
    std::invoke([]() { print_num(42); });    // invoke a lambda

    const Foo foo(314159);
    std::invoke(&Foo::print_add, foo, 1);    // invoke a member function

    std::invoke(PrintNum{}, 18);             // invoke a function object

     // "invoke" (access) a data member
    std::cout << "num_: " << std::invoke(&Foo::num_, foo) << '\n';
```

- Приклад використання в уніфікованому коді:

```
template <typename T, typename F>
void CallOnRangeInvoke(T& container, F f)
{
    for (auto&& elem : container)
        std::invoke(f, elem);
}
```

  зможе як почергово передати елементи масиву callable f, так і викликати метод для кожного із об'єктів, вказівник на який передано у аргументі f.

- ◇ Приклад взято із: `https://www.bfilipek.com/2020/10/understanding-invoke.html`.
- Курйоз – invoke() можна застосовувати до вказівників на члени даних, тоді вона еквівалентна розіменуванню: `https://devblogs.microsoft.com/oldnewthing/20220401-00/?p=106426` – "The std::invoke function does more than invoke functions".
- Детальніше: `https://en.cppreference.com/w/cpp/utility/functional/invoke`

std::apply – C++17

- Викликає callable із кортежем аргументів, розпакувавши його[26]:

```
int add(int first, int second) { return first + second; }

template<typename... Ts>
std::ostream& operator<<(std::ostream& os,
                         std::tuple<Ts...> const& theTuple) {
    std::apply (
        [&os](Ts const&... tupleArgs)          {
            os << '[';
            std::size_t n{0};
            ((os << tupleArgs << (++n != sizeof...(Ts) ? ", " : "")),
            os << ']';
        }, theTuple
    );
    return os;
}
        ..............................
std::cout << std::apply(add, std::pair(1, 2)) << '\n';
std::tuple myTuple(25, "Hello", 9.31f, 'c');
std::cout << myTuple << '\n';
```

[26] Приклад звідси: https://en.cppreference.com/w/cpp/utility/apply.

- "What does f(x) mean in C++?" – не так страшно, як з ініціалізацією, але на диво багато значень...
- "Why can't std::apply figure out which overload I intend to use? Only one of then will work!" – нагадування, що у місці виклику компілятор не знає про потребу виконати вибір перевантаження, а там де реалізація – вже немає іншої частини інформації.

